

```
from fastapi import FastAPI, HTTPException
```

```
app = FastAPI()
```

```
policy = TrustPolicy(  
    trusted_issuers=["did:example:issuer1"],  
  
    required_types=["UniversityDegreeCredential"],  
    min_assurance_level=2,  
    allowed_claims=["name", "degree",  
"graduationYear"]  
)
```

```
demo_cred = CredentialRecord(  
    id="cred-123",  
    issuer="did:example:issuer1",  
    types=["VerifiableCredential",  
"UniversityDegreeCredential"],  
    subject_claims={  
        "name": "Alice",  
        "degree": "MSc Computer Science",  
        "graduationYear": 2024,
```

```
        "gpa": 4.0
    },
    proof={"type": "Ed25519Signature2020",
"jwt": "demo-proof"},
    issuance_date="2025-01-10",
    assurance_level=2
)
```

```
@app.post("/wallet/present")
def wallet_present():
    consent = ConsentRecord(
        holder_did="did:example:holder1",
        verifier_id="verifier-service",
        requested_claims=["name", "degree",
"graduationYear"],
        approved_claims=["name", "degree"],
        purpose="job_application",
        timestamp="2026-03-30T12:00:00Z"
    )
    try:
        vp = build_presentation(demo_cred,
policy, consent)
```

```
        return vp
    except ValueError as e:
        raise HTTPException(status_code=400,
detail=str(e))
```

```
@app.post("/verifier/verify")
def verifier_verify(presentation: Dict[str, Any]):
    result = verify_presentation(presentation,
policy)
    if not result["accepted"]:
        raise HTTPException(status_code=400,
detail=result)
    return result
```